

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»

Лабораторная работа №6
Многопоточное вычисление интегралов функций

Выполнила:

Иванова Елизавета Вадимовна,
студентка группы 6301-030301D

Самара 2025

Задание 1. Численное интегрирование методом трапеций

В классе Functions реализован статический метод:

```
public static double integral(Function function, double leftX, double rightX, double dx)
```

- Метод разбивает отрезок $[a, b]$ на подынтервалы длиной dx (последний — остаточный).
- На каждом подынтервале вычисляется площадь трапеции.
- Добавлена проверка: если a или b выходят за границы области определения функции — выбрасывается исключение.
- Для функции $f(x)=e^x$ на $[0, 1]$ установлено: при $dx=10^{-8}$ достигается точность до 7 знака после запятой.

Задание 2. Последовательная реализация (nonThread)

- Создан метод nonThread(), формирующий 100 заданий:
 - функция: логарифм с основанием $\in [1, 10]$, $\in [1, 10]$,
 - $x \in [0, 100]$, $x \in [0, 100]$, $x \in [100, 200]$, $x \in [100, 200]$, $dx \in [0, 1]$, $dx \in [0, 1]$.
- Все задания обрабатываются последовательно, без потоков.
- Вывод: 100 пар строк Source ... / Result

```
Source 3.4639214417989783 149.11799506607093 0.3201182858481265
Result 3.4639214417989783 149.11799506607093 0.3201182858481265 308.5401679272782
Source 71.53755333836148 146.5878658513106 0.18275223072908586
Result 71.53755333836148 146.5878658513106 0.18275223072908586 197.1227449612441
Source 25.295857766477457 199.97621849681335 0.2166702543425404
Result 25.295857766477457 199.97621849681335 0.2166702543425404 355.77628373000584
Source 11.913873111413565 100.99298373912717 0.2050679235188384
Result 11.913873111413565 100.99298373912717 0.2050679235188384 159.39839034992394
Source 20.42096748483233 100.9156052031406 0.06746266663077172
Result 20.42096748483233 100.9156052031406 0.06746266663077172 346.05538956237825
Source 68.62683404645415 187.0928153615557 0.37294370261218734
Result 68.62683404645415 187.0928153615557 0.37294370261218734 328.03162231111537
Source 70.50639063162298 182.48929522463396 0.8925236200107971
Result 70.50639063162298 182.48929522463396 0.8925236200107971 345.11328185596386
Source 40.150193838441275 192.27324614224457 0.845895303735841
Result 40.150193838441275 192.27324614224457 0.845895303735841 473.93031162921926
Source 45.51582254140867 169.98416921982943 0.9598126528591292
Result 45.51582254140867 169.98416921982943 0.9598126528591292 280.40992337092416
Source 85.46959082177207 152.4781805090302 0.44847686151855715
```

```

Result 72.15298731782055 181.75577662982695 0.18096830371568418 327.8179853094831
Source 0.26386373696005716 161.99214236912508 0.2567569487712623
Result 0.26386373696005716 161.99214236912508 0.2567569487712623 411.9138605909037
Source 15.986535195327123 187.5321425301697 0.0867133423072306
Result 15.986535195327123 187.5321425301697 0.0867133423072306 683.3822241500665
Source 56.71259815972005 170.56186892809308 0.17117017267606904
Result 56.71259815972005 170.56186892809308 0.17117017267606904 252.75095085538237
Source 35.027074142706674 119.03929817160764 0.8681859071276997
Result 35.027074142706674 119.03929817160764 0.8681859071276997 640.5513648838898
Source 46.396469828119166 144.0934906199504 0.26984137079982007
Result 46.396469828119166 144.0934906199504 0.26984137079982007 443.18522164216176
Source 33.17634526233654 158.95221164649246 0.31081413016166815
Result 33.17634526233654 158.95221164649246 0.31081413016166815 4089.6562468248553
Source 18.27153799475001 102.1891042083287 0.9930504194365093
Result 18.27153799475001 102.1891042083287 0.9930504194365093 165.54983930295762
Source 60.161069484820615 197.86671207633452 0.8847680297242987
Result 60.161069484820615 197.86671207633452 0.8847680297242987 317.4733753341486
Source 22.503703207505097 173.06179312079792 0.8576881544148293
Result 22.503703207505097 173.06179312079792 0.8576881544148293 317.3834363847641

Process finished with exit code 0

```

→ Все 100 заданий обработаны, вывод корректен.

Задание 3. Простая многопоточная реализация (simpleThreads)

- Реализованы классы:
 - SimpleGenerator (генератор),
 - SimpleIntegrator (вычислитель).
- Оба потока синхронизируются на объекте Task через:
 - synchronized (task),
 - wait() / notify().
- Проблема: при первом запуске наблюдались:
 - NullPointerException (интегратор читал null),
 - "перемешивание" параметров (чтение половины данных от одного задания, половины — от другого).
- Решение:

Генератор ждёт, пока function == null, затем заполняет все поля.

Интегратор ждёт, пока function != null, читает всё, затем сбрасывает function = null и уведомляет.

```
Source 10.346634853025726 121.32655428132921 0.04921020753714611
Result 10.346634853025726 121.32655428132921 0.04921020753714611 228.8765158131456
Source 83.57039028812206 175.7364562817822 0.3302193294250442
Result 83.57039028812206 175.7364562817822 0.3302193294250442 197.76163668209983
Source 40.7083162398177 185.0185875531751 0.1758573549456528
Result 40.7083162398177 185.0185875531751 0.1758573549456528 731.6922195644058
Source 89.50090149667001 162.2658254056796 0.4006805024359441
Result 89.50090149667001 162.2658254056796 0.4006805024359441 177.13698338078547
Source 48.25004654238172 147.00328678173878 0.5254339071464702
Result 48.25004654238172 147.00328678173878 0.5254339071464702 278.7680185294461
Source 7.349656582177422 102.96876357272976 0.3134167536414545
Result 7.349656582177422 102.96876357272976 0.3134167536414545 231.21045258447603
Source 2.030775611641622 102.648643667155 0.34002852530505
Result 2.030775611641622 102.648643667155 0.34002852530505 231.46484414828234
Source 13.170014789611372 100.50104991926652 0.07697225732461044
Result 13.170014789611372 100.50104991926652 0.07697225732461044 2176.334367440647
Source 63.888955425768316 189.0831228652641 0.3720046978567553
Result 63.888955425768316 189.0831228652641 0.3720046978567553 1589.4818705854582
```

```
Result 51.40983589646062 197.7106510129722 0.6532124460546567 444.4373494124777
Source 64.52067246201403 180.6220566151104 0.8293640610414913
Result 64.52067246201403 180.6220566151104 0.8293640610414913 557.544682182005
Source 15.734844709011552 199.4142651421201 0.8625008235973997
Result 15.734844709011552 199.4142651421201 0.8625008235973997 478.0237133558946
Source 52.85188546729266 142.88887400274425 0.48205697961043825
Result 52.85188546729266 142.88887400274425 0.48205697961043825 252.72863062591625
Source 12.912948189566077 196.54285809534434 0.511056212466545
Result 12.912948189566077 196.54285809534434 0.511056212466545 961.3474377183396
Source 94.95977511996189 163.95682021232352 0.5510457251945721
Result 94.95977511996189 163.95682021232352 0.5510457251945721 311.65426861435344
Source 55.41566849530201 175.84816510073424 0.9363567119509285
Result 55.41566849530201 175.84816510073424 0.9363567119509285 514.596101135883
Source 37.663798797514644 135.0099716844689 0.04268244386209219
Result 37.663798797514644 135.0099716844689 0.04268244386209219 3743.1487404785908
Source 79.5270717297258 168.8915689377029 0.2468793252682141
Result 79.5270717297258 168.8915689377029 0.2468793252682141 373.4827592962122
Source 77.60766533814429 113.23694557305876 0.5780610616901194
Result 77.60766533814429 113.23694557305876 0.5780610616901194 103.43882679552846
```

```
Process finished with exit code 0
```

Задание 4. Реализация с собственным семафором (complicatedThreads)

- Создан класс SimpleSemaphore — бинарный семафор на основе wait()/notify().
- Изначально использовался один семафор, но это привело к гонке: оба потока конкурировали за захват, и данные перезаписывались.
- Решение: применена классическая схема Producer–Consumer с двумя семафорами:
 - empty — сигнал, что Task пуст и можно писать,
 - full — сигнал, что Task заполнен и можно читать.
- В main() семафор full изначально занят, чтобы интегратор ждал первую запись.
- Результат:

Строго поочерёдный вывод Source → Result,
все задания уникальны,
нет потери данных,
программа устойчива к изменению приоритетов

В соответствии с заданием: основной поток ждёт 50 мс, затем вызывает interrupt() для обоих потоков.

```

Source 68.35204431781402 149.43121934893753 0.5293680626242693
Result 68.35204431781402 149.43121934893753 0.5293680626242693 435.59166459524795
Source 75.34929629065566 175.82418340028676 0.8035040511209253
Result 75.34929629065566 175.82418340028676 0.8035040511209253 307.5225456791428
Source 40.539328286080355 190.69012549031052 0.5921329517785466
Result 40.539328286080355 190.69012549031052 0.5921329517785466 327.62246397646294
Source 89.79134631582882 177.28582576108764 0.0633602101557248
Result 89.79134631582882 177.28582576108764 0.0633602101557248 196.88059217689522
Source 95.59288994077752 140.10756154808766 0.9858845943047991
Result 95.59288994077752 140.10756154808766 0.9858845943047991 502.7853746378117
Source 42.873614707772845 144.05846867119214 0.22726348179111366
Result 42.873614707772845 144.05846867119214 0.22726348179111366 376.85085412643537
Source 90.85046573035356 150.5566453142609 0.7220506371392962
Result 90.85046573035356 150.5566453142609 0.7220506371392962 153.81667329783141
Source 38.514443081541636 158.5215659118826 0.4474345812394048
Result 38.514443081541636 158.5215659118826 0.4474345812394048 253.61366625477777
Source 19.4456183576177 105.14912214252128 0.7652949703711202
Result 19.4456183576177 105.14912214252128 0.7652949703711202 191.6189620291048
Source 22.834451777494813 106.4831807168535 0.34213302432850834
Result 22.834451777494813 106.4831807168535 0.34213302432850834 2092.7848133763823
Source 61.90388779132206 121.01430375678929 0.742121565075504
Result 61.90388779132206 121.01430375678929 0.742121565075504 122.72937804482378
Result 38.514443081541636 158.5215659118826 0.4474345812394048 253.61366625477777
Source 19.4456183576177 105.14912214252128 0.7652949703711202
Result 19.4456183576177 105.14912214252128 0.7652949703711202 191.6189620291048
Source 22.834451777494813 106.4831807168535 0.34213302432850834
Result 22.834451777494813 106.4831807168535 0.34213302432850834 2092.7848133763823
Source 61.90388779132206 121.01430375678929 0.742121565075504
Result 61.90388779132206 121.01430375678929 0.742121565075504 122.72937804482378
Source 88.36804921453363 100.88383124440574 0.7035590997123824
Result 88.36804921453363 100.88383124440574 0.7035590997123824 26.391357236969903
Source 82.12984081805715 154.7442592406386 0.055301140727924136
Result 82.12984081805715 154.7442592406386 0.055301140727924136 161.1268455290761
Source 29.111137420324773 183.9244449363425 0.20728176484128336
Result 29.111137420324773 183.9244449363425 0.20728176484128336 1424.7215305378027
Source 36.925591031612534 169.2708953439115 0.15802667639971735
Result 36.925591031612534 169.2708953439115 0.15802667639971735 283.42156471906804
Source 28.04562680012087 179.8839674439878 0.00209854881045346
Generator was interrupted
Result 28.04562680012087 179.8839674439878 0.00209854881045346 406.2041869032582
Integrator was interrupted

Process finished with exit code 0

```

Порядок вывода сообщений "Generator was interrupted" и "Integrator was interrupted" может варьироваться, поскольку потоки завершаются асинхронно. Однако все сгенерированные задания корректно обрабатываются, что подтверждается строгим соответствием параметров в каждой паре Source/Result

Выводы

- Метод трапеций успешно реализован и протестирован.
- Примитивы wait()/notify() позволяют организовать чередование потоков, но требуют строгого контроля состояния (function = null).

- Использование собственного семафора обеспечивает надёжную синхронизацию.
- Схема с двумя семафорами (empty/full) — наиболее надёжное решение для задачи "один производитель — один потребитель".
- Все режимы работы (nonThread, simpleThreads, complicatedThreads) протестированы и корректны.